HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

BỘ MÔN THỰC TẬP CƠ SỞ



BÁO CÁO TIẾN ĐỘ TUẦN 2

KHỞI TẠO BACKEND, DATABASE VÀ XÁC THỰC

Đề tài: XÂY DỰNG WEBSITE HỖ TRỢ HỌC TẬP CÁ NHÂN HÓA BẰNG AI

Giảng viên hướng dẫn : **ThS.Nguyễn Xuân Đức**

Họ và tên sinh viên : **Trần Đỗ Lan Phương**

Mã sinh viên : **B23DCCN667**

Lớp : **D23CQCN09-B**

Hà Nội – 2026

I. THÔNG TIN CHUNG

Sinh viên: Trần Đỗ Lan Phương | Mã sinh viên: B23DCCN667 | Lớp: D23CQCN09-B

Giảng viên hướng dẫn: ThS. Nguyễn Xuân Đức

Tuần báo cáo: Tuần 2 — Giai đoạn Khởi tạo Backend, Database và Xác thực

Đề tài: Xây dựng website hỗ trợ học tập cá nhân hóa bằng AI.

II. MỤC TIÊU TUẦN 2

Trên cơ sở các thiết kế đã hoàn thành ở tuần 1, tuần 2 chuyển sang giai đoạn cài đặt phần backend với các mục tiêu chính:

- Khởi tạo dự án backend với Node.js 20, Express 4, TypeScript và cấu trúc thư mục theo mô hình MVC + Service Layer.
- Cấu hình kết nối MongoDB qua Mongoose 8, hỗ trợ cả mode kết nối thật và mock in-memory cho phát triển nhanh.
- Định nghĩa schema và model Mongoose cho 6 collection chính (users, subjects, questions, assessments, learningpaths, practicesessions).
- Cài đặt module xác thực: đăng ký, đăng nhập, đổi mật khẩu, đổi email với bcrypt cost 12 và JWT 7 ngày.
- Xây dựng auth middleware xác minh Bearer token cho mọi endpoint bảo mật.
- Tích hợp Zod để validate input theo schema, trả về lỗi rõ ràng cho client.
- Tích hợp Swagger UI để tài liệu hóa API và kiểm thử trực tiếp trên trình duyệt.
- Tích hợp Gemini provider qua interface IAIProvider (Dependency Inversion).

III. KẾT QUẢ THỰC HIỆN

1 Khởi tạo dự án backend

Dự án backend được khởi tạo trong thư mục backend/ với cấu trúc rõ ràng theo mô hình MVC mở rộng (Model — Controller — Service — Route — Middleware), giúp tách biệt nghiệp vụ khỏi tầng giao tiếp HTTP và tầng dữ liệu.

Cấu trúc thư mục backend/src:

- src/app.ts — khởi tạo Express, đăng ký middleware toàn cục (CORS, JSON parser), gắn Swagger UI tại /api-docs và đăng ký 7 router con.

• src/server.ts — khởi động HTTP server, đọc PORT từ biến môi trường.

• src/config/ — cấu hình database (Mongoose connect), Swagger spec, biến môi trường.

• src/models/ — 6 file định nghĩa schema Mongoose tương ứng 6 collection.

• src/controllers/ — 7 file controller, mỗi controller xử lý request và gọi service.

• src/services/ — 8 file service chứa toàn bộ business logic, tách biệt với controller.

• src/routes/ — 7 file router định tuyến URL.

• src/middleware/ — auth.middleware.ts xác minh JWT.

• src/types/ — TypeScript interfaces dùng chung toàn project.

Các phụ thuộc chính (package.json):

• express ^4.18.3 — web framework.

• mongoose ^8.2.0 — ODM cho MongoDB.

• bcryptjs ^2.4.3 — băm mật khẩu (cost 12).

• jsonwebtoken ^9.0.2 — sinh và xác minh JWT.

• zod ^3.22.4 — validate input theo schema.

• cors, dotenv — quản lý CORS và biến môi trường.

• swagger-ui-express, swagger-jsdoc — tự động sinh tài liệu API.

• @google/genai ^1.52.0 — SDK chính thức của Google Gemini.

• typescript ^5.3.3, ts-node-dev ^2.0.0 — phát triển TypeScript có hot-reload.

2 Cấu hình kết nối MongoDB

Module config/database.ts thiết lập kết nối MongoDB qua Mongoose với hai chế độ: kết nối thật (đọc URI từ biến môi trường MONGODB_URI) hoặc mock in-memory (khi MONGODB_URI=mock — dùng cho phát triển khi chưa có MongoDB local). Chế độ mock giúp test luồng auth ngay cả khi không có database.

Điểm chính:

• Sử dụng strictQuery, autoIndex theo môi trường, retry tự động khi mất kết nối.

• Áp dụng index theo thiết kế ở mục 3.4 báo cáo tuần 1: email unique cho users, code unique cho subjects, compound { topicIds, difficulty } cho questions, single-guard index cho assessments.

• Tạo file scripts/seed.ts để nạp dữ liệu mẫu (4 môn học mặc định: Toán THPT, Tiếng Anh THPT, Lập trình Python, Tiếng Anh giao tiếp).

3 Định nghĩa các Mongoose Models

Sáu Mongoose models đã được implement đầy đủ theo thiết kế ở báo cáo tuần 1. Mỗi model định nghĩa schema (kèm validators), virtual fields, instance methods và static methods cần thiết.

• **User:** Tài khoản người dùng — email unique, passwordHash, profile (name, grade, avatarUrl), learningGoals[] (subjectId, targetLevel, targetDate), currentLevel (level + weakTopics). Có method comparePassword(candidate) gói bcrypt.compare.

• **Subject:** Môn học — code unique, name, mảng topics nhúng (Topic subdocument có _id riêng).

• **Question:** Câu hỏi — topicIds[], difficulty (1-3), content, options[], correctAnswer, explanation, usageCount, metadata.generatedBy.

• **Assessment:** Bài đánh giá đầu vào — userId, subjectId, mảng questions (questionId + questionSnapshot + userAnswer + isCorrect), score, status (in_progress/submitted).

• **LearningPath:** Lộ trình học — userId, subjectId, basedOnAssessmentId, mảng milestones (week, topicIds, difficulty, estimatedDays, resources).

• **PracticeSession:** Phiên luyện tập thích ứng — userId, questions[], adaptiveState (currentDifficulty, consecutiveCorrect, consecutiveWrong), results.

4 Cài đặt module xác thực (Auth)

Module Auth gồm 4 thành phần: AuthService (logic), AuthController/Route (HTTP layer), auth.middleware.ts (kiểm tra Bearer token), và Zod schemas (validate input). Mật khẩu được băm bằng bcrypt với cost 12. JWT có hạn 7 ngày, secret đọc từ biến môi trường JWT_SECRET.

Các endpoint đã cài đặt:

• POST /api/auth/register — đăng ký tài khoản. Validate {email, password, name, grade}. Kiểm tra email chưa tồn tại, băm mật khẩu, lưu user, trả về { token, user }.

• POST /api/auth/login — đăng nhập. Validate {email, password}. Tìm user, so sánh mật khẩu, sinh JWT, trả về { token, user }.

• PATCH /api/auth/password — đổi mật khẩu (cần JWT). Xác minh mật khẩu cũ qua bcrypt.compare, băm mật khẩu mới cost 12, update.

• PATCH /api/auth/email — đổi email (cần JWT). Xác minh mật khẩu, kiểm tra email mới chưa tồn tại, update.

• GET /api/auth/me — lấy thông tin user hiện tại từ JWT.

Auth middleware:

Middleware authMiddleware đọc header Authorization, tách Bearer token, gọi authService.verifyToken để xác minh, gắn payload vào req.user. Middleware optionalAuthMiddleware cho phép truy cập không bắt buộc đăng nhập (dùng cho các endpoint công khai có behavior thay đổi theo user).

5 Kiểm thử API với Swagger UI

Swagger UI được mount tại /api-docs giúp xem tài liệu API và gửi request thử trực tiếp trên trình duyệt mà không cần Postman. Mỗi route được chú thích bằng JSDoc @swagger để swagger-jsdoc tự sinh OpenAPI spec.

Quy trình kiểm thử:

• Mở http://localhost:5000/api-docs → giao diện Swagger.

• Test POST /api/auth/register với payload mẫu → nhận JWT.

• Click "Authorize" trên Swagger UI, dán JWT → mọi request sau đó tự động gắn Bearer token.

• Test các endpoint bảo mật (đổi mật khẩu, lấy hồ sơ) → xác nhận middleware hoạt động đúng.